

Geospatial models Serving and Benchmarking with vLLM

Goal: Prepare and **serve** a
TerraTorch models with **vLLM**
and **benchmark** performance

Agenda

- Step 1: Serve a TerraTorch model with vLLM
- Step 2: Benchmark the inference server with *vLLM bench* and *GuideLLM*
- Step 3: Primer on how to scale vLLM in Kubernetes
- Bonus step: Customize vLLM with IOProcessors

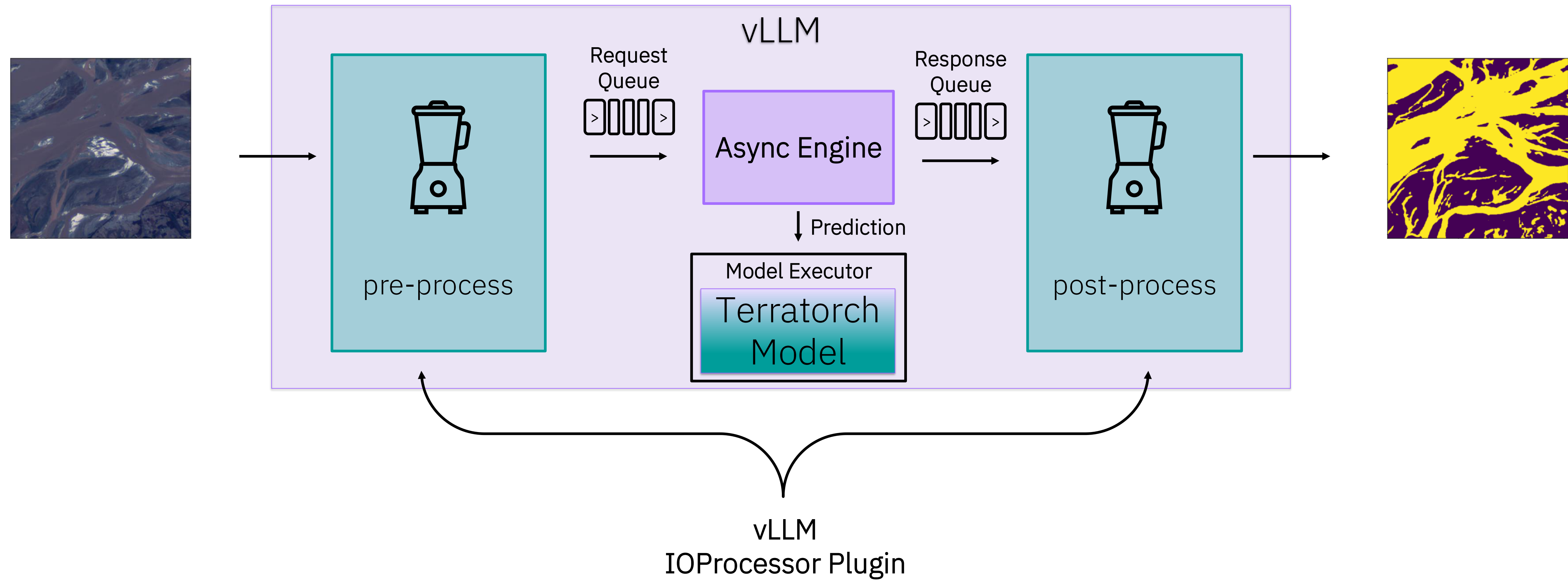
vLLM

vLLM is an [open-source](#) library to [high-performance inference and serving](#) of foundation models.

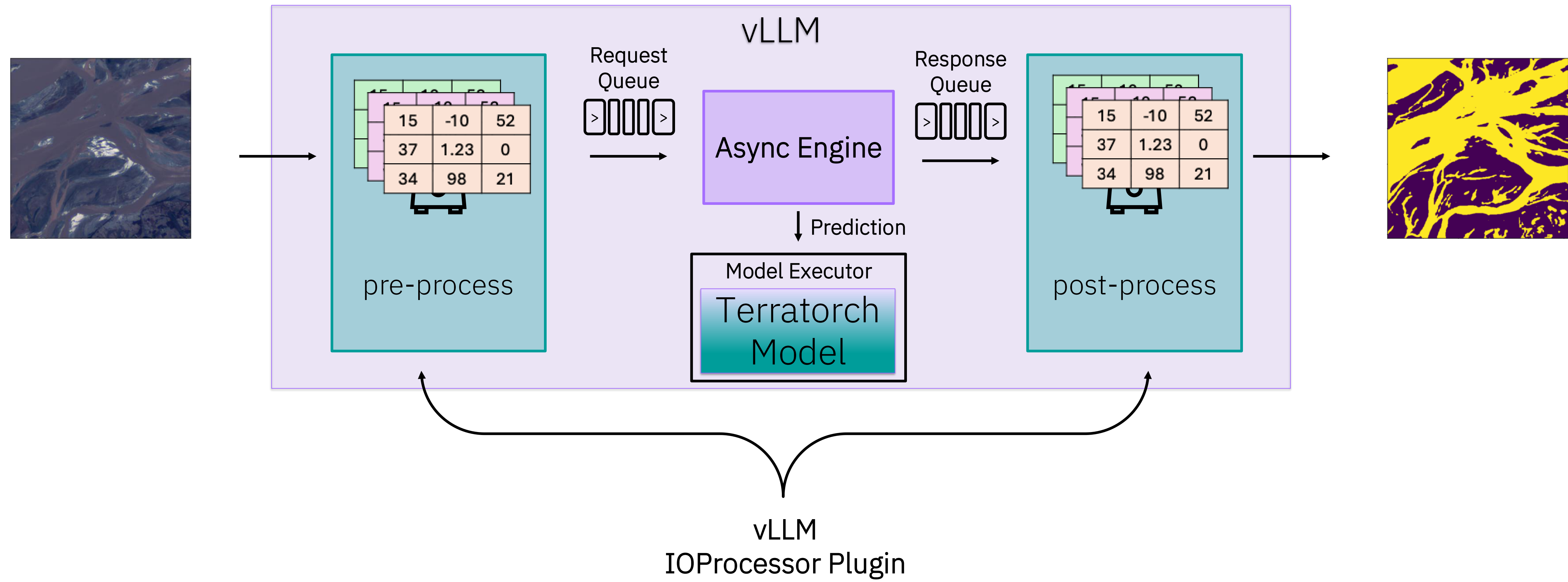
- Standard [REST API](#) for easy integration
- High GPU utilisation via continuous batching & scheduling
- Community-driven: [extensible, transparent](#), less lock-in
- Cloud-native ready: fits [Kubernetes](#) deployments
- Benchmarking tools: vllm bench, GuideLLM for [latency/throughput](#)



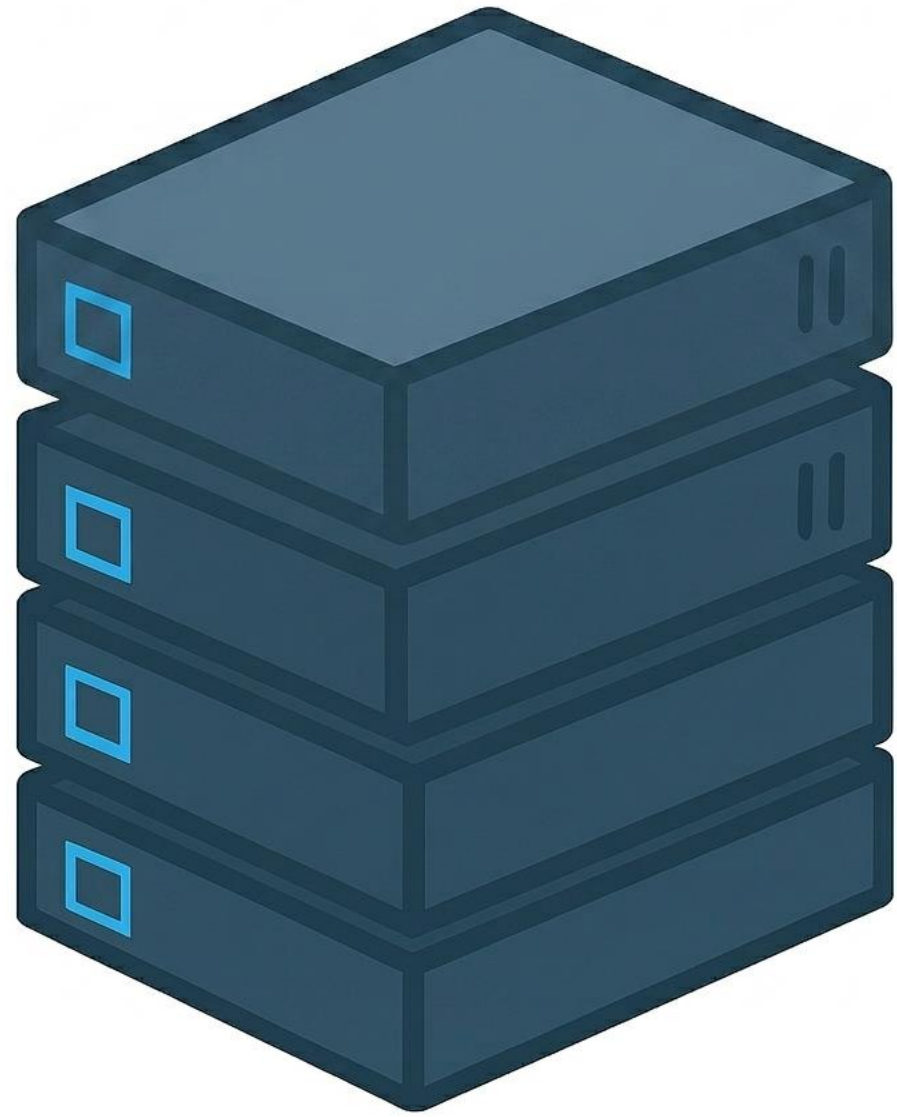
vLLM inference for EO models



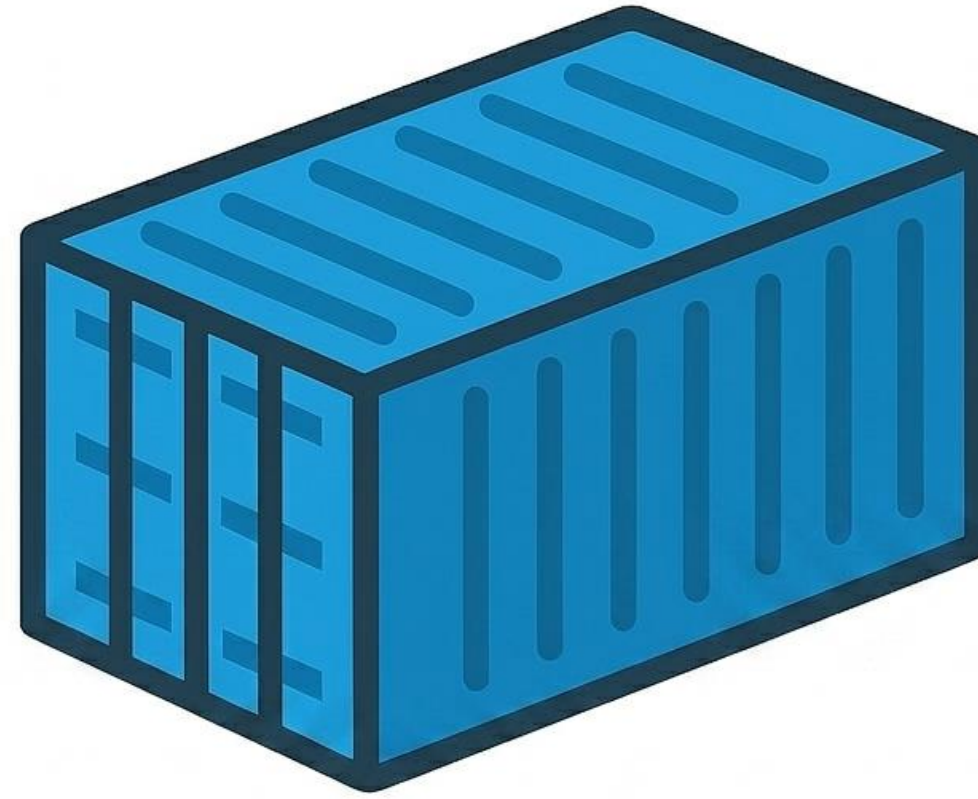
vLLM inference for EO models



Storing models



Filesystem



Containers



HuggingFace

Storing models

 **Hugging Face**

Search models, datasets, users...

Models

 **mgazz** / **prithvi-eo-flood**

like 0

License: apache-2.0

Model card

Files and versions

xet

Community

Settings

main

prithvi-eo-flood

391 MB

 **mgazz**

Upload 2 files

4454e57

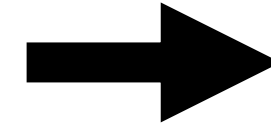
VERIFIED

 .gitattributes	Safe	1.52 kB	
 README.md	Safe	28 Bytes	
 config_deploy.yaml		4.42 kB	
 state_dict.ckpt		391 MB	xet 

Prepare Model

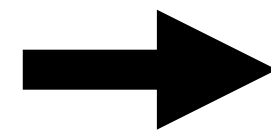


model.ckpt



state_dict.bin

config_deploy.yaml



config.json

config.json

```
{  
  "architectures": [  
    "Terratorch"  
  ],  
  "pretrained_cfg": {  
    .....
```

Reference to the
Model Executor

```
    "data": {  
      "class_path": "terratorch.datamodules.GenericNonGeoSegmentationDataModule",  
      .....
```

Pre-process
Input

```
    "model": {  
      "class_path": "terratorch.tasks.SemanticSegmentationTask",  
      .....
```

Type of Model

```
    "input": {  
      "target": "pixel_values",  
      "data": {  
        "pixel_values": { "type": "torch.Tensor", "shape": [6,512,512]}  
      }  
    }  
  }  
}
```

Input validation
and vLLM
warm-up

What to expect

```
sagemaker-user@default:~/ESA-NASA-Workshop-2026/Day 2/Track 3/Operational-GeoAI-Model-Serving/Part2/1_run_model_in_vllm$ \
> ls -lah
total 746M
drwxr-xr-x. 4 sagemaker-user users 4.0K May 19 13:25 .
drwxr-xr-x. 9 sagemaker-user users 4.0K May 19 13:25 ..
drwxr-xr-x. 3 sagemaker-user users 25 May 19 13:21 .cache
drwxr-xr-x. 2 sagemaker-user users 6 May 19 13:20 .ipynb_checkpoints
-rw-r--r--. 1 sagemaker-user users 6.4K May 19 13:12 README.md
-rw-r--r--. 1 sagemaker-user users 5.9K May 19 13:25 config.json
-rw-r--r--. 1 sagemaker-user users 4.4K May 19 13:21 config_deploy.yaml
-rwxr-xr-x. 1 sagemaker-user users 3.1K May 19 13:12 convert_ckpt_to_bin.py
-rw-r--r--. 1 sagemaker-user users 559K May 19 13:12 inference.ipynb
-rw-r--r--. 1 sagemaker-user users 13K May 19 13:12 mask.tiff
-rw-r--r--. 1 sagemaker-user users 373M May 19 13:25 state_dict.bin
-rw-r--r--. 1 sagemaker-user users 373M May 19 13:21 state_dict.ckpt
-rw-r--r--. 1 sagemaker-user users 3.0K May 19 13:12 vllm_config_generator.py
```


Model Preparation Time!

(follow `1_run_model_in_vllm/README.md`)

Serve the model with vLLM

```
vllm serve 1_run_model_in_vllm \
  --skip-tokenizer-init \
  --enable-mm-embeds \
  --max-num-seqs 32 \
  --io-processor-plugin \
  terratorch_segmentation \
  --enforce-eager
```

Serve the model with vLLM

```
vllm serve 1_run_model_in_vllm \ #Model Name (path)
  --skip-tokenizer-init \ #No Tokenizer for EO
  --enable-mm-embeds \ #Turn on multi-modal
  --max-num-seqs 32 \ #Control parallelism
  --io-processor-plugin \
  terratorch_segmentation \ #IOProc Selection
  --enforce-eager #Disable CUDA graph
```


What to expect

```
299]
299]
299]  version 0.19.1
299] model 1_run_model_in_vllm
299]
299]
233] non-default args: {'model_tag': '1_run_model_in_vllm', 'model': '1_run_model_in_vllm',
e, 'io_processor_plugin': 'terratorch_segmentation', 'enable_mm_embeds': True, 'max_num_seqs
```


What to expect

299]

299]

299]

299]

299]

299]

233] non-default args: {'model_t

e, 'io_processor_plugin': 'terrat



```
19 13:32:52 [api_server.py:596] Starting vLLM server on http://0.0.0.0:8000
19 13:32:52 [launcher.py:37] Available routes are:
19 13:32:52 [launcher.py:46] Route: /openapi.json, Methods: GET, HEAD
19 13:32:52 [launcher.py:46] Route: /docs, Methods: GET, HEAD
19 13:32:52 [launcher.py:46] Route: /docs/oauth2-redirect, Methods: GET, HEAD
19 13:32:52 [launcher.py:46] Route: /redoc, Methods: GET, HEAD
19 13:32:52 [launcher.py:46] Route: /tokenize, Methods: POST
19 13:32:52 [launcher.py:46] Route: /detokenize, Methods: POST
19 13:32:52 [launcher.py:46] Route: /load, Methods: GET
19 13:32:52 [launcher.py:46] Route: /version, Methods: GET
19 13:32:52 [launcher.py:46] Route: /health, Methods: GET
19 13:32:52 [launcher.py:46] Route: /metrics, Methods: GET
19 13:32:52 [launcher.py:46] Route: /v1/models, Methods: GET
19 13:32:52 [launcher.py:46] Route: /ping, Methods: GET
19 13:32:52 [launcher.py:46] Route: /ping, Methods: POST
19 13:32:52 [launcher.py:46] Route: /invocations, Methods: POST
19 13:32:52 [launcher.py:46] Route: /pooling, Methods: POST
Started server process [1357]
Waiting for application startup.
Application startup complete.
```


Serve the model with vLLM

```
vllm serve mgazz/prithvi-eo-flood-vLLM \
  --skip-tokenizer-init \
  --enable-mm-embeds \
  --max-num-seqs 32 \
  --io-processor-plugin terratorch_segmentation \
  --enforce-eager
```

Model Serving and Inference Time!

(continue with the instructions in the README.md
and run the example in inference.ipynb)

Benchmark Tools

Key questions:

- **Baseline Performance**
 - What is the latency/throughput running an inference with **vLLM**?
- **Scalability behaviour**
 - How does the throughput/latency of the vLLM service change as we **increase clients requests**?
- **Production Reliability (SLO)**
 - How should the inference stack be designed to **meet latency and throughput SLOs**?
 - What configurations ensure **predictable performance under load**?

Benchmark Tools

	vLLM Bench	Guidellm
Organization	vLLM	vLLM
Location	vLLM project	Standalone project
Best for	Single-point measure Regression tests	Performance sweep Custom arrival patterns

Jsonl Dataset entries

```
{  
  "prompt": {  
    "data": {  
      "data": ".../samples/India_900498_S2Hand.tif",  
      "data_format": "path",  
      "out_data_format": "b64_json",  
      "indices": [1,2,3,8,11,12]  
    }, ...  
  }  
}
```

Jsonl Dataset entries

```
{  
  "prompt": {  
    "data": {  
      "data": ".../samples/India_900498_S2Hand.tif",  
      "data_format": "path",  
      "out_data_format": "b64_json",  
      "indices": [1,2,3,8,11,12]  
    }, ...  
  }  
}
```


vllm bench

```
vllm bench serve \  
  --base-url http://localhost:8000 \      #Endpoint  
  --dataset-name=custom \                #Name of the model  
  --model 1_run_model_in_vllm \          #skip tokenizer (EO models do not use it)  
  --skip-tokenizer-init \                #server endpoint  
  --endpoint /pooling \                  #request handler  
  --backend vllm-pooling \                #additional end-to-end latency metric  
  --percentile-metrics e2el \            #percentiles  
  --metric-percentiles 25,75,99 \        #number of requests  
  --num-prompts 100 \                    #request rate (poisson default)  
  --request-rate 10 \                    #datasetpath  
  --dataset-path ./dataset_path_input_india.jsonl
```

vllm bench

```
INFO 05-15 15:45:05 [datasets.py:305] oversampled requests to reach 100
Starting initial single prompt test run...
Skipping endpoint ready check.
Starting main benchmark run...
Traffic request rate: 10.0
Burstiness factor: 1.0 (Poisson process)
Maximum request concurrency: None
100%|████████████████████████████████████████| 100/100 [00:10<00:00, 9.85it/s]
===== Serving Benchmark Result =====
Successful requests:          100
Failed requests:              0
Request rate configured (RPS): 10.00
Benchmark duration (s):       10.15
Total input tokens:           0
Request throughput (req/s):    9.85
-----End-to-end Latency-----
Mean E2EL (ms):               195.60
Median E2EL (ms):              167.47
P25 E2EL (ms):                 117.79
P75 E2EL (ms):                 262.21
P99 E2EL (ms):                 474.87
=====
```

GuideLLM sweep

```
guidellm benchmark \
  --target http://localhost:8000 \           #server endpoint
  --backend openai_http \                   #inference scenario (alternative: vllm_python)
  --model 1_run_model_in_vllm \            #model name
  --data dataset_path_input_india_guidellm.jsonl \ #dataset
  --request-format /pooling \               #format request for /pooling endpoint
  --data-column-mapper pooling_column_mapper \. #surface the correct key from dataset
  --max-requests 100 \                     # number of requests
  --output-path results.json                # save output to file
```


What to expect

Benchmarks									
[14:37:32]	100% synchronous	(complete)	Req:	11.2 req/s,	0.09s Lat,	1.0 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:37:43]	100% throughput@512	(complete)	Req:	18.8 req/s,	3.10s Lat,	58.4 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:37:50]	100% constant@12.17	(complete)	Req:	12.1 req/s,	0.13s Lat,	1.5 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:00]	100% constant@13.12	(complete)	Req:	13.1 req/s,	0.13s Lat,	1.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:09]	100% constant@14.07	(complete)	Req:	14.0 req/s,	0.12s Lat,	1.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:18]	100% constant@15.02	(complete)	Req:	14.9 req/s,	0.12s Lat,	1.8 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:26]	100% constant@15.96	(complete)	Req:	15.9 req/s,	0.12s Lat,	1.9 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:35]	100% constant@16.91	(complete)	Req:	16.6 req/s,	0.18s Lat,	2.9 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:42]	100% constant@17.86	(complete)	Req:	17.3 req/s,	0.21s Lat,	3.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:50]	100% constant@18.81	(complete)	Req:	17.8 req/s,	0.33s Lat,	6.0 Conc,	100 Comp,	0 Inc,	0 Err
			Tok:	0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen

Generating...

GuideLLM sweep

Benchmarks			throughput	Mean latency	Mean concurrency	Requests completed		
			↓	↓	↓	↓		
[14:37:32]	100% synchronous	(complete)	Req: 11.2 req/s,	0.09s Lat,	1.0 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:37:43]	100% throughput@512	(complete)	Req: 18.8 req/s,	3.10s Lat,	58.4 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:37:50]	100% constant@12.17	(complete)	Req: 12.1 req/s,	0.13s Lat,	1.5 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:00]	100% constant@13.12	(complete)	Req: 13.1 req/s,	0.13s Lat,	1.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:09]	100% constant@14.07	(complete)	Req: 14.0 req/s,	0.12s Lat,	1.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:18]	100% constant@15.02	(complete)	Req: 14.9 req/s,	0.12s Lat,	1.8 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:26]	100% constant@15.96	(complete)	Req: 15.9 req/s,	0.12s Lat,	1.9 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:35]	100% constant@16.91	(complete)	Req: 16.6 req/s,	0.18s Lat,	2.9 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:42]	100% constant@17.86	(complete)	Req: 17.3 req/s,	0.21s Lat,	3.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:50]	100% constant@18.81	(complete)	Req: 17.8 req/s,	0.33s Lat,	6.0 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen

Generating...

GuideLLM sweep

Rates			throughput	Mean latency	Mean concurrency	Requests completed		
			↓	↓	↓	↓		
Benchmarks								
[14:37:32]	100% synchronous	(complete)	Req: 11.2 req/s,	0.09s Lat,	1.0 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:37:43]	100% throughput@512	(complete)	Req: 18.8 req/s,	3.10s Lat,	58.4 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:37:50]	100% constant@12.17	(complete)	Req: 12.1 req/s,	0.13s Lat,	1.5 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:00]	100% constant@13.12	(complete)	Req: 13.1 req/s,	0.13s Lat,	1.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:09]	100% constant@14.07	(complete)	Req: 14.0 req/s,	0.12s Lat,	1.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:18]	100% constant@15.02	(complete)	Req: 14.9 req/s,	0.12s Lat,	1.8 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:26]	100% constant@15.96	(complete)	Req: 15.9 req/s,	0.12s Lat,	1.9 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:35]	100% constant@16.91	(complete)	Req: 16.6 req/s,	0.18s Lat,	2.9 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:42]	100% constant@17.86	(complete)	Req: 17.3 req/s,	0.21s Lat,	3.7 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen
[14:38:50]	100% constant@18.81	(complete)	Req: 17.8 req/s,	0.33s Lat,	6.0 Conc,	100 Comp,	0 Inc,	0 Err
			Tok: 0.0 gen/s,	0.0 tot/s,	0.0ms TTFT,	0.0ms ITL,	0 Prompt,	0 Gen

Generating...

Benchmarking Time!

(follow [2_vllm_benchmarking/README.md](#))

Customize your benchmark with GuideLLM profiles

Profile	Goal	Best for
sweep	rate exploration	finding limits
throughput	max load	peak throughput
concurrent	# users	user-level latency
constant	fixed RPS	steady-state
poisson	randomized RPS	realistic traffic
synchronous	sequential	baseline latency

Recap so far

We learned how to:

- Prepare a TerraTorch model and serve it with vLLM
- Benchmark models with vLLM bench and GuideLLM
- Find the sweet spot between Throughput and Latency with GuideLLM